Stereo Vision Face Depth Estimation

Using the Intel RealSense D455F Dual-IR Camera

A technical report on stereo vision, disparity estimation, and real-time face depth measurement

Akash Dhande

CS 5097 — Mixed Reality

Prof. Ryan Bockmon

Camera	Intel RealSense D455F (Dual IR Stereo)
Methods	Hardware Depth • OpenCV SGBM • Custom SAD
Face detector	Google BlazeFace (MediaPipe TFLite)
Date	April 23, 2026

Contents

1 Introduction and Project Overview **2**

2 Background Concepts **2**
2.1 The Pinhole Camera Model . . . 2
2.2 Stereo Vision Fundamentals . . . 3
2.3 Disparity and Depth Relationship . . . 3
2.4 Stereo Rectification . . . 4
2.5 Infrared Imaging for Stereo . . . 4
2.6 Block Matching for Stereo . . . 4

3 Intel RealSense D455F Camera **4**
3.1 Hardware Architecture . . . 5
3.2 Sensor Specifications . . . 5
3.3 pyrealsense2 SDK Integration . . . 5

4 Face Detection with BlazeFace **6**
4.1 MediaPipe BlazeFace Architecture . . . 6
4.2 Inference Pipeline . . . 7

5 Depth Estimation Approaches **8**
5.1 Approach 1 — Hardware Depth (Intel ASIC) . . . 8
5.2 Approach 2 — Semi-Global Block Matching (SGBM) . . . 9
5.3 Approach 3 — Custom SAD Block Matching . . . 10
5.4 Disparity to Depth Conversion . . . 12

6 System Architecture and Processing Pipeline **13**

7 Visualisation System **14**

8 Comparison of Approaches **16**

9 Dependencies and Libraries **17**

10 How to Run **17**
10.1 Hardware Requirements . . . 17
10.2 System-Level Dependency: librealsense2 . . . 17
10.3 Python Environment Setup . . . 18
10.4 Project File Checklist . . . 18
10.5 Running the Application . . . 18
10.6 Common Issues . . . 19

11 Conclusion **19**

1. Introduction and Project Overview

This project implements a real-time face depth estimation system using an **Intel RealSense D455F** depth camera. The system captures four synchronized streams simultaneously — an RGB colour stream, a hardware depth stream, and two infrared (IR) streams — and uses them to detect human faces and compute the distance of each detected face from the camera using three independent methods.

The core motivation is to compare three fundamentally different depth estimation strategies operating on the same input data in real time:

- **Hardware Depth (Intel ASIC):** The D455F's onboard stereo processor computes depth maps at the hardware level using Intel's proprietary, FPGA-accelerated algorithm. This provides the highest quality reference depth map.
- **Semi-Global Block Matching (SGBM):** A classical computer vision algorithm implemented in OpenCV. It uses dynamic programming along multiple directions to produce smooth, high-quality disparity maps from the raw IR stereo pair.
- **Sum of Absolute Differences (SAD) Block Matching:** A custom NumPy implementation of the simplest stereo matching algorithm. It performs a brute-force search over all possible disparities for every pixel, optimised using the integral image (cumulative sum) technique.

All three methods are visualised simultaneously in a real-time **six-panel grid**, with face bounding boxes and depth measurements overlaid on each panel. This allows direct visual comparison of the quality, noise characteristics, and accuracy of each approach.

2. Background Concepts

2.1. The Pinhole Camera Model

A camera is modelled mathematically as a **pinhole camera**: a 3D world point $P = (X, Y, Z)$ is projected onto the 2D image plane at pixel coordinates (u, v) through the camera's focal point. The projection is governed by the **intrinsic matrix** K:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix}, \qquad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{1}$$

Parameter	Meaning	Typical value (D455F, 640×480)
f_x	Focal length in x (pixels)	$\approx$ 380–420 px
f_y	Focal length in y (pixels)	$\approx$ 380–420 px
c_x	Principal point x	$\approx$ 320 px
c_y	Principal point y	$\approx$ 240 px

2.2. Stereo Vision Fundamentals

Stereo vision mimics human binocular depth perception. Two cameras, separated by a known **baseline distance** B, observe the same scene from slightly different viewpoints. Because the cameras are displaced horizontally, a 3D point in the scene projects to *different horizontal positions* in the two image planes. This horizontal shift is called **disparity**.

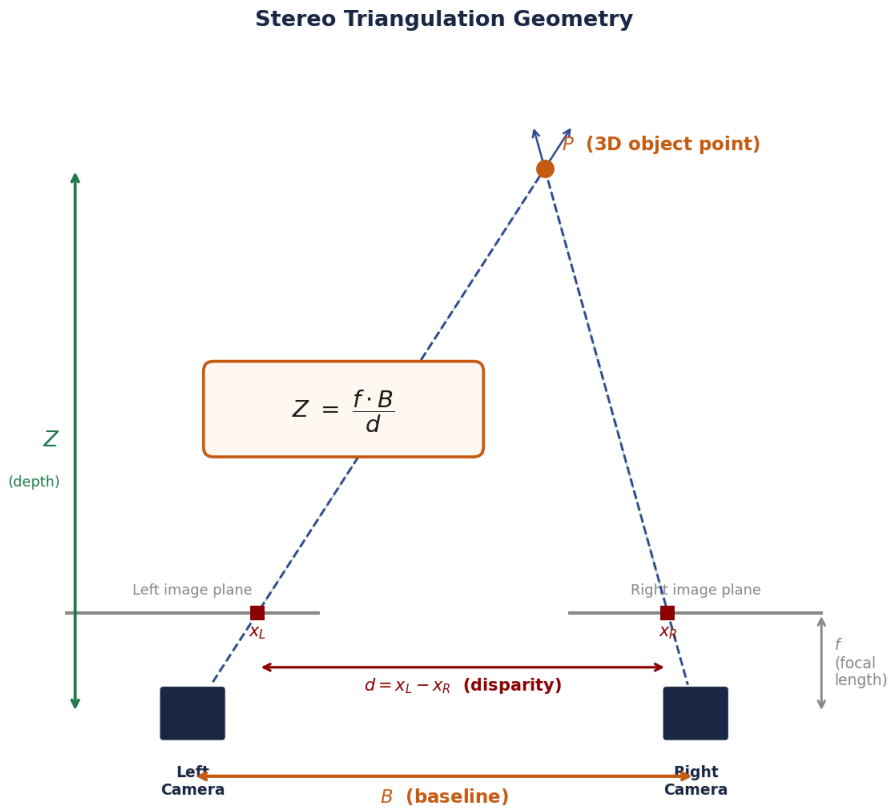

Figure 1: Stereo triangulation geometry. The same 3D point P projects to different horizontal positions x_L and x_R in the left and right image planes. The horizontal shift $d = x_L - x_R$ is the disparity, and depth is recovered as $Z = f \cdot B \,/\, d$.

2.3. Disparity and Depth Relationship

For a **rectified stereo pair** (where both cameras share the same horizontal scan lines), the relationship between disparity and depth is given by the stereo triangulation formula:

$$Z = \frac{f_x \cdot B}{d} \tag{2}$$

where Z = depth (metres), f_x = focal length in pixels, B = baseline in metres, d = disparity in pixels.

Key observation: Depth is *inversely proportional* to disparity. Closer objects have larger disparity; distant objects have smaller disparity. At zero disparity (infinity), depth is undefined — this is why stereo cameras have a maximum useful range.

This formula is used identically for both the SGBM and SAD approaches in this project:

```
# fx and baseline extracted once at startup from camera
    intrinsics/extrinsics
depth_metres = (fx * baseline) / disparity_pixels
```

Listing 1: Disparity to depth conversion (live_stream.py)

2.4. Stereo Rectification

In a real stereo camera, the two cameras are never perfectly aligned. **Rectification** applies homographic transformations to both images so that corresponding points lie on the *same horizontal scan line*. After rectification, stereo matching reduces from a 2D search to a 1D search along rows, dramatically reducing computation.

> **Note:** The IR streams from the D455F are pre-rectified by the camera firmware. The SGBM and SAD algorithms in this project can therefore directly search along horizontal rows without any additional rectification step.

2.5. Infrared Imaging for Stereo

Visible-light stereo matching struggles in textureless or uniformly lit regions because block matching requires local image texture to find correspondences. The D455F addresses this with an **IR dot projector** that floods the scene with a random dot pattern in the near-infrared spectrum (≈850 nm), invisible to the human eye. This creates artificial texture on featureless surfaces (walls, clothing), dramatically improving stereo matching reliability.

The two IR cameras capture this textured scene as 8-bit grayscale images (Y8 format, single channel). These IR images are the direct input to both the SGBM and SAD algorithms.

2.6. Block Matching for Stereo

Block matching is the fundamental operation underlying most local stereo matching algorithms. For each pixel (y, x) in the left image:

1. Extract a small $W \times W$ patch (block) centred at (y, x).
2. Search along the same horizontal row in the right image for disparities 0 to D.
3. Compute a **similarity metric** between the left block and each right-image candidate block.
4. Assign the disparity d^* that minimises the dissimilarity metric.

Common metrics include SAD (Sum of Absolute Differences), SSD (Sum of Squared Differences), and NCC (Normalized Cross-Correlation). This project uses **SAD** for the custom implementation. Block size W controls a key trade-off: larger blocks are more robust to noise but lose precision near depth discontinuities; smaller blocks preserve edges but are noisier.

3. Intel RealSense D455F Camera

3.1. Hardware Architecture

The Intel RealSense D455F is a factory-calibrated stereo depth camera designed for depth sensing up to ∼6 metres. It integrates multiple sensors on a single board:

Component	Description	Role in this project
IR Left Camera	Global shutter, Y8 format	Left image for SGBM & SAD
IR Right Camera	Global shutter, Y8 format	Right image for SGBM & SAD
RGB Camera	Colour, BGR8 format	Face detection input
IR Dot Projector	850 nm near-IR pattern	Adds texture for stereo matching
Stereo ASIC	Intel Vision Processor D4	Hardware depth map (Z16)
IMU	Accelerometer + Gyroscope	Not used in this project

The physical layout from left to right on the camera bar is: IR Left Camera — IR Dot Projector — RGB Camera — IR Right Camera. The baseline between the two IR cameras is approximately **95 mm**.

3.2. Sensor Specifications

Parameter	Value
Stereo Baseline	≈95 mm (between IR cameras)
Depth Range	0.4 m – 6 m (indoor)
IR Image Format	Y8 (8-bit grayscale, single channel)
Colour Image Format	BGR8 (8-bit per channel, 3 channels)
Depth Output Format	Z16 (16-bit unsigned, millimetres)
Frame Rate (this project)	30 FPS on all four streams
Resolution (this project)	640×480 on all four streams
Interface	USB 3.1 Gen 1 (5 Gbps)

3.3. pyrealsense2 SDK Integration

The camera is controlled through Intel's **pyrealsense2** Python SDK. The four streams are configured and started once at program launch, and the camera intrinsics (focal length) and extrinsics (baseline) are extracted immediately after:

```python
import pyrealsense2 as rs

pipeline = rs.pipeline()
config   = rs.config()
config.enable_stream(rs.stream.color,    640, 480, rs.format.bgr8, 30)
config.enable_stream(rs.stream.depth,    640, 480, rs.format.z16, 30)
config.enable_stream(rs.stream.infrared, 1, 640, 480, rs.format.y8, 30)
config.enable_stream(rs.stream.infrared, 2, 640, 480, rs.format.y8, 30)
pipeline.start(config)

# Extract focal length and baseline from camera parameters
profile      = pipeline.get_active_profile()
```

```
13 ir1           = profile.get_stream(rs.stream.infrared, 1)
14 ir2           = profile.get_stream(rs.stream.infrared, 2)
15 ir_intrinsics = ir1.as_video_stream_profile().get_intrinsics()
16 fx        = ir_intrinsics.fx                                # focal
       length (pixels)
17 baseline = abs(ir1.get_extrinsics_to(ir2).translation[0])   #
       ~0.095 m
```

Listing 2: Camera setup and intrinsics extraction (live_stream.py lines 1–26)

Finding the focal length and baseline: Intel does not publicly document the exact focal length or stereo baseline of the D455F in any official datasheet or product page. Searching online yields only approximate physical measurements and community estimates. The precise, calibrated values are embedded in the camera's firmware and are only accessible programmatically at runtime through the pyrealsense2 SDK — as shown in the code above via `get_intrinsics()` and `get_extrinsics_to()`. This is the only reliable source for these parameters and is the approach used in this project.

In the main loop, `pipeline.wait_for_frames()` blocks until all four enabled streams deliver a frame at the same timestamp, guaranteeing synchronisation:

```
1  frames          = pipeline.wait_for_frames()
2  color_frame     = frames.get_color_frame()
3  depth_frame     = frames.get_depth_frame()
4  ir_left_frame   = frames.get_infrared_frame(1)
5  ir_right_frame  = frames.get_infrared_frame(2)
6
7  color_image = np.asanyarray(color_frame.get_data())    # (H, W, 3)
       uint8
8  ir_left     = np.asanyarray(ir_left_frame.get_data()) # (H, W)
         uint8
9  ir_right    = np.asanyarray(ir_right_frame.get_data()) # (H, W)
         uint8
10
11 # Hardware depth at a specific pixel (in metres)
12 distance_m  = depth_frame.get_distance(cx, cy)
```

Listing 3: Synchronised frame capture and array conversion

4. Face Detection with BlazeFace

4.1. MediaPipe BlazeFace Architecture

Face detection is performed using **BlazeFace**, a lightweight neural network developed by Google as part of the MediaPipe framework. BlazeFace is specifically designed for real-time, mobile-first face detection.

Key architectural features:

- **Anchor-based detection:** Uses 896 pre-defined anchor boxes optimised for the

face size range. The "full range" model variant (used here) supports faces from 20% to 100% of the image width.

- **Depthwise separable convolutions:** MobileNet-style separable convolutions minimise parameter count while maintaining accuracy.
- **Weighted NMS:** Non-maximum suppression merges overlapping candidate detections into a single clean bounding box per face.
- **TFLite format:** The model file `blaze_face_full_range.tflite` is 1.1 MB, making it efficient for CPU inference.

4.2. Inference Pipeline

The detector is initialised once using MediaPipe's Tasks API and reused across all frames. The `IMAGE` running mode processes one frame at a time:

```python
import mediapipe as mp

BaseOptions          = mp.tasks.BaseOptions
FaceDetector         = mp.tasks.vision.FaceDetector
FaceDetectorOptions  = mp.tasks.vision.FaceDetectorOptions
VisionRunningMode    = mp.tasks.vision.RunningMode

options = FaceDetectorOptions(
    base_options=BaseOptions(model_asset_path="blaze_face_full_range.tflite"),
    running_mode=VisionRunningMode.IMAGE,
    min_detection_confidence=0.5,
)

with FaceDetector.create_from_options(options) as detector:
    mp_image = mp.Image(
        image_format=mp.ImageFormat.SRGB,
        data=cv2.cvtColor(bgr_frame, cv2.COLOR_BGR2RGB),
    )
    results = detector.detect(mp_image)

    for detection in results.detections:
        bbox  = detection.bounding_box
        x, y, w, h = bbox.origin_x, bbox.origin_y, bbox.width, bbox.height
        score = detection.categories[0].score   # confidence in [0, 1]
```

Listing 4: Face detector initialisation and inference (live_stream.py)

Once face bounding boxes are known, the centre of each box is used to sample depth values from all three depth maps. A small x-offset (`x_offset_factor=0.2`) shifts the sampling point 20% of the bounding box width to the right, to compensate for typical face pose bias:

```python
def draw_detections_with_depth(image, detections, get_depth_fn, unit,
```

```
                                    x_offset_factor=0.2):
    for (x, y, w, h, _) in detections:
        x  = x + int(x_offset_factor * w)        # slight
           rightward offset
        cx = min(x + w // 2, image.shape[1] - 1)
        cy = min(y + h // 2, image.shape[0] - 1)
        depth = get_depth_fn(cx, cy)
        cv2.rectangle(image, (x, y), (x + w, y + h), color, 2)
        if depth > 0:
            put_text_with_bg(image, f"{depth:.2f}{unit}", (x, y -
                4), ...)
```

Listing 5: Depth sampling at face centre (live_stream.py lines 62–71)

5. Depth Estimation Approaches

5.1. Approach 1 — Hardware Depth (Intel ASIC)

The D455F contains a dedicated **stereo depth ASIC** (Intel RealSense Vision Processor D4) that processes the two IR streams onboard. It implements a highly optimised stereo matching pipeline in hardware, including:

- Stereo rectification
- Disparity computation with sub-pixel interpolation
- Temporal and spatial filtering
- Hole-filling for invalid depth regions

The resulting depth map is output as a **Z16 stream** (16-bit unsigned integer, values in millimetres). Accessing hardware depth requires only a single SDK call:

```
depth_frame  = frames.get_depth_frame()
distance_m   = depth_frame.get_distance(cx, cy)   # direct depth
   in metres

# Visualisation: scale Z16 values to 8-bit and apply JET
   colourmap
depth_image = np.asanyarray(depth_frame.get_data())
depth_color = cv2.applyColorMap(
    cv2.convertScaleAbs(depth_image, alpha=0.03), cv2.
       COLORMAP_JET
)
```

Listing 6: Hardware depth access and visualisation

> The hardware depth map serves as the ground-truth reference in this project. It benefits from temporal filtering, sub-pixel accuracy, and proprietary sensor calibration that software-only approaches cannot match.

5.2. Approach 2 — Semi-Global Block Matching (SGBM)

Semi-Global Matching (SGM), proposed by Hirschmüller (2005), extends local block matching with a global smoothness constraint enforced via dynamic programming along multiple directions. OpenCV's `StereoSGBM` is the standard CPU implementation.

5.2.1. Algorithm Steps

1. **Cost computation:** For each pixel (y, x) and each candidate disparity $d \in [0, D)$, compute a matching cost over the 11×11 window centred at that pixel.

2. **Semi-global aggregation:** Propagate costs along 4 or 8 directions (paths) using dynamic programming. Each path minimises:

$$L_r(p, d) = C(p, d) + \min \begin{cases} L_r(p - r, d) \\ L_r(p - r, d \pm 1) + P_1 \\ \min_i L_r(p - r, i) + P_2 \end{cases} \tag{3}$$

   where $P_1 < P_2$ are smoothness penalties. P_1 penalises small disparity changes (± 1 pixel); P_2 penalises larger jumps, encouraging piecewise-smooth depth surfaces.

3. **Winner-takes-all:** For each pixel, select $d^* = \arg\min_d \sum_r L_r(p, d)$.

4. **Post-processing:** Apply left-right consistency check, uniqueness ratio filter, and speckle removal.

5.2.2. Implementation

```python
def compute_stereo_depth(ir_left, ir_right):
    block_size = 11
    stereo = cv2.StereoSGBM_create(
        minDisparity     = 0,
        numDisparities   = 128,        # search range: 0 to 127 px
        blockSize        = 11,         # 11x11 matching window
        P1               = 8  * block_size**2,    # = 968
        P2               = 32 * block_size**2,    # = 3872
        disp12MaxDiff    = 1,          # left-right consistency tolerance (px)
        uniquenessRatio = 10,          # best match must be 10% better than 2nd
        speckleWindowSize = 100,       # connected-region size for noise removal
        speckleRange     = 32,         # max disparity variation in a speckle
        mode = cv2.STEREO_SGBM_MODE_SGBM_3WAY,
    )
    # OpenCV returns Q4 fixed-point (multiply-by-16); divide to get pixels
    disparity = stereo.compute(ir_left, ir_right).astype(np.float32) / 16.0
    colormap  = cv2.applyColorMap(
        255 - cv2.convertScaleAbs(disparity, alpha=255.0 / 128.0),
        cv2.COLORMAP_JET)
```

```
    return colormap, disparity
```

Listing 7: SGBM stereo matching (live_stream.py lines 124–143)

5.2.3. Parameter Reference

Parameter	Value	Meaning
`numDisparities`	128	Horizontal search range (0–127 pixels)
`blockSize`	11	Matching window size (11×11 pixels)
`P1`	968	Penalty for ±1 disparity change; encourages smooth surfaces
`P2`	3872	Penalty for larger jumps; must satisfy $P_2 > P_1$
`disp12MaxDiff`	1	Max allowable left–right disparity difference (pixels)
`uniquenessRatio`	10	Best match must be 10% better than second-best
`speckleWindowSize`	100	Min region size to keep; smaller regions are invalidated
`speckleRange`	32	Max disparity variation within a valid region
`mode`	`SGBM_3WAY`	Optimised variant using 3-way path aggregation

Fixed-point output: OpenCV's SGBM returns disparity as a 16-bit integer in Q4 fixed-point format (i.e., actual disparity × 16). Dividing by 16.0 recovers the true disparity in pixels, with 1/16-pixel sub-pixel resolution.

5.3. Approach 3 — Custom SAD Block Matching

The **Sum of Absolute Differences (SAD)** is the simplest stereo similarity metric. For two $W \times W$ blocks A and B:

$$\mathrm{SAD}(A, B) = \sum_{i=0}^{W-1} \sum_{j=0}^{W-1} |A(i, j) - B(i, j)| \tag{4}$$

5.3.1. Algorithm

For each disparity $d \in [0, 128)$:

1. Shift the right image d pixels to the right (so pixel (y, x) in the left image corresponds to pixel $(y, x - d)$ in the right image).
2. Compute the per-pixel absolute difference between left and shifted right.
3. Sum differences over every 15×15 block using the **integral image trick** (see below).
4. For each pixel, keep the disparity d^* that yields the minimum SAD.

5.3.2. Integral Image Optimisation

Naively computing a $W \times W$ block sum requires $O(W^2)$ operations per pixel per disparity. The **integral image (summed area table)** reduces this to $O(1)$ after an $O(H \times W)$ prefix-sum pass:

```
abs_diff = np.abs(left - shifted_right)    # per-pixel absolute
    difference
```

```python

# Step 1: row-wise prefix sum
row_cs   = np.cumsum(abs_diff, axis=0)
row_sums = row_cs[B-1:] - np.concatenate(
    [np.zeros((1, W)), row_cs[:-B]], axis=0)
# row_sums[i, j] = sum of abs_diff[i : i+B, j]

# Step 2: column-wise prefix sum on row_sums
col_cs   = np.cumsum(row_sums, axis=1)
block_sad = col_cs[:, B-1:] - np.concatenate(
    [np.zeros((H-B+1, 1)), col_cs[:, :-B]], axis=1)
# block_sad[i, j] = sum of abs_diff over the B x B block at (i, j
    )

# Step 3: update best-disparity map
update           = sad_map < min_sad
min_sad[update]  = sad_map[update]
disparity[update]= float(d)
```

Listing 8: Cumulative-sum block matching (live_stream.py lines 99–116)

This approach processes all pixels at each disparity level in a single vectorised NumPy pass, rather than looping over individual pixels.

5.3.3. Full Implementation

```python
def compute_sad_depth(ir_left, ir_right):
    block_size     = 15
    num_disparities= 128
    half_b = block_size // 2
    h, w   = ir_left.shape
    left   = ir_left.astype(np.float32)
    right  = ir_right.astype(np.float32)

    min_sad   = np.full((h, w), np.inf, dtype=np.float32)
    disparity = np.zeros((h, w),  dtype=np.float32)

    for d in range(num_disparities):
        shifted_right        = np.zeros_like(right)
        shifted_right[:, d:] = right[:, :w - d]

        abs_diff = np.abs(left - shifted_right)

        row_cs   = np.cumsum(abs_diff, axis=0)
        row_sums = row_cs[block_size-1:] - np.concatenate(
            [np.zeros((1, w)), row_cs[:-block_size]], axis=0)

        col_cs   = np.cumsum(row_sums, axis=1)
        block_sad= col_cs[:, block_size-1:] - np.concatenate(
            [np.zeros((h-block_size+1, 1)), col_cs[:, :-
               block_size]], axis=1)

        sad_map  = np.full((h, w), np.inf, dtype=np.float32)
```

```
        sad_map[half_b:h-half_b, half_b:w-half_b] = block_sad

        update            = sad_map < min_sad
        min_sad[update]   = sad_map[update]
        disparity[update]= float(d)

    colormap = cv2.applyColorMap(
        255 - cv2.convertScaleAbs(disparity, alpha=255.0/
           num_disparities),
        cv2.COLORMAP_JET)
    return colormap, disparity
```

Listing 9: Complete SAD depth computation (live_stream.py lines 79–121)

5.3.4. Interactive Pedagogical Visualiser

The file `stereo_matching.html` (included in the project directory) is a self-contained browser tool designed to make the SAD block-matching algorithm tangible and observable step by step.

What it shows:

- A synthetic stereo image pair is rendered in the browser — a scene with three coloured rectangular objects at different depths, with per-pixel noise added to simulate real sensor texture.
- Clicking anywhere on the **left image** triggers an animated search: the 11×11 reference patch slides along the **epipolar row** in the right image, testing every candidate disparity from 0 to 35 pixels.
- A **live bar chart** updates frame by frame showing the SAD cost at each candidate position, coloured by state — checked (blue), current (orange), and best match (green).
- After the search completes, the tool reports x_L, x_R, and the resulting disparity $d = x_L - x_R$, and highlights the winning patch in the right image.

How to use: Open `stereo_matching.html` directly in any modern browser — no server or dependencies required. Click different regions of the left image (background, orange object, blue object) to see how disparity changes with simulated depth, and how the SAD cost curve looks in textureless versus textured areas.

5.4. Disparity to Depth Conversion

Both SGBM and SAD produce disparity maps in pixels. Converting to depth in metres uses the stereo triangulation formula with the focal length and baseline extracted at startup:

```
# SGBM depth at detected face centre
lambda cx, cy: (fx * baseline / stereo_disparity[cy, cx])
               if stereo_disparity[cy, cx] > 0 else 0

# SAD depth at detected face centre
```

```
lambda cx, cy: (fx * baseline / sad_disparity[cy, cx])
               if sad_disparity[cy, cx] > 0 else 0

# Hardware depth (no formula needed)
depth_frame.get_distance(cx, cy)   # returns metres directly
```

Listing 10: Depth readout per face (live_stream.py lines 185–199)

The guard `if disparity > 0` prevents division by zero for pixels where stereo matching failed (disparity = 0 indicates an invalid match).

6. System Architecture and Processing Pipeline

Each frame in the main loop follows this end-to-end sequence:

1. **Frame capture.** `pipeline.wait_for_frames()` blocks until all four streams deliver a synchronised frameset at 30 FPS.
2. **Array conversion.** Each frame is converted to a NumPy array. IR frames are $(H \times W)$ grayscale; colour and depth frames are $(H \times W \times 3)$ or $(H \times W)$ respectively.
3. **Hardware depth colourmap.** The raw Z16 depth array is scaled and coloured with the JET colourmap for visualisation.
4. **IR to BGR conversion.** Both IR grayscale frames are converted to 3-channel BGR using `cv2.COLOR_GRAY2BGR` so they can be stacked with the colour panels.
5. **SGBM computation.** `compute_stereo_depth(ir_left, ir_right)` runs OpenCV SGBM on the IR stereo pair and returns a disparity map and colourmap.
6. **SAD computation.** `compute_sad_depth(ir_left, ir_right)` runs the custom NumPy SAD algorithm on the same IR pair.
7. **Face detection.** `detect_faces(color_image, detector)` runs the BlazeFace model on the RGB frame and returns a list of `(x, y, w, h, score)` tuples.
8. **Depth overlay.** `draw_detections_with_depth()` overlays face bounding boxes and depth text on all three depth visualisation panels.
9. **Grid assembly and display.** The six panels are assembled into a 2×3 grid and displayed in a single OpenCV window.

```
top_row  = np.hstack((color_image, depth_colormap,
   stereo_colormap))
bot_row  = np.hstack((ir_left_bgr, ir_right_bgr,  sad_colormap))
combined = np.vstack((top_row, bot_row))
cv2.imshow("Stream Grid", combined)   # 1920 x 960 px display
```

Listing 11: Grid assembly (live_stream.py lines 208–212)

7. Visualisation System

The live output is a single OpenCV window showing all six data streams simultaneously in a 2×3 **grid layout**. Each panel is 640×480 pixels, giving an overall display of 1920×960 pixels.

RGB Colour (face detection overlay)	Intel D455F Device Depth (hardware Z16)	SGBM Stereo Depth (OpenCV SGBM)
IR Left (stream 1, Y8)	IR Right (stream 2, Y8)	SAD Depth (custom NumPy)

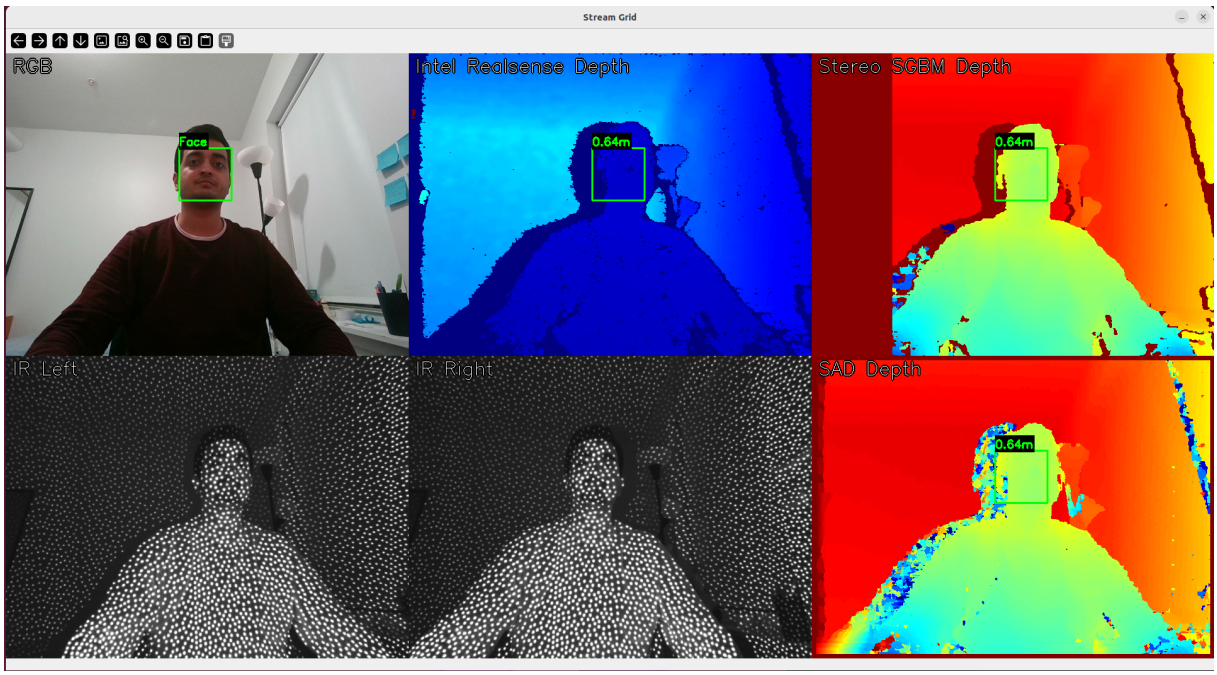

Figure 2: Live output of the running application. The six-panel grid shows (top row, left to right): RGB with detected face and bounding box; Intel RealSense hardware depth map reporting 0.64 m; OpenCV SGBM depth map reporting 0.64 m. Bottom row: IR Left and IR Right streams showing the structured IR dot pattern projected onto the scene; custom SAD depth map reporting 0.64 m. All three depth methods agree on the face distance in this frame.

All three depth panels map depth to the **JET colourmap** such that **near objects appear blue/cool** and **distant objects appear red/warm**. In JET, low values map to blue and high values map to red.

The **hardware depth** panel achieves this naturally: Z16 depth values increase with distance, so closer pixels produce smaller values and appear blue:

```
# small depth = near = small value = blue; large depth = far =
    red
depth_color = cv2.applyColorMap(
    cv2.convertScaleAbs(depth_image, alpha=0.03), cv2.
        COLORMAP_JET)
```

Listing 12: Hardware depth colourmap — direct JET mapping

The **SGBM and SAD** panels work in disparity space, where closer objects have *larger* disparity. To restore the same near-blue/far-red convention, the disparity value is **inverted** (subtracted from 255) before applying JET:

```
# large disparity = near; after inversion: small value = blue (near)
display  = 255 - cv2.convertScaleAbs(disparity, alpha=255.0 / max_disparity)
colormap = cv2.applyColorMap(display, cv2.COLORMAP_JET)
```

Listing 13: Inverted JET colourmap for disparity panels

Panel labels and depth readouts are drawn with a solid black background rectangle to ensure readability over both dark and bright depth regions:

```
def put_text_with_bg(image, text, pos, font_scale, text_color, bg_color=(0,0,0)):
    font = cv2.FONT_HERSHEY_SIMPLEX
    (tw, th), baseline = cv2.getTextSize(text, font, font_scale, thickness=1)
    x, y = pos
    cv2.rectangle(image, (x, y-th-baseline), (x+tw, y+baseline), bg_color, cv2.FILLED)
    cv2.putText(image, text, pos, font, font_scale, text_color, thickness=1)
```

Listing 14: Text with background helper (live_stream.py lines 35–40)

8. Comparison of Approaches

Characteristic	Hardware ASIC	OpenCV SGBM	Custom SAD
Algorithm	Proprietary Intel firmware	Semi-global DP	Local block matching
Block size	Internal (unknown)	11×11 px	15×15 px
Search range	Up to 1280 px	128 disparities	128 disparities
Global smoothing	Yes (temporal + spatial)	Yes (semi-global)	No
Post-processing	Yes (many filters)	Speckle + L-R check	None
Sub-pixel acc.	Yes (hardware)	Yes (÷16)	No (integer only)
Noise level	Very low	Low–medium	High (blocky)
Edge quality	Good	Moderate	Poor
CPU compute	N/A (on-device)	∼5–20 ms/frame	∼100–500 ms/frame
Code complexity	1 SDK call	∼15 lines	∼40 lines

Key observations:

- **Hardware depth** is the most reliable. It benefits from temporal filtering, sub-pixel interpolation, and proprietary optimisation for the D455F sensor geometry.
- **SGBM** produces significantly smoother results than SAD. The dynamic programming smoothness penalties (P_1, P_2) discourage abrupt disparity changes, reducing noise. It is the best software alternative to hardware depth.
- **SAD** produces the noisiest depth maps. Without any global smoothness constraint, each pixel independently picks the best-matching disparity, leading to salt-and-pepper noise and incorrect matches in textureless regions. However, it is the most transparent algorithm for understanding the fundamentals of stereo matching.
- Despite the SAD implementation using the integral image trick for vectorised computation, iterating over 128 disparities sequentially in Python still makes it the slowest of the three methods.

9. Dependencies and Libraries

Library	Version	Purpose and key APIs
`pyrealsense2`	2.x	Camera control and frame capture. Key APIs: `rs.pipeline`, `rs.config`, `enable_stream`, `wait_for_frames`, `get_distance`, `get_intrinsics`, `get_extrinsics_to`.
`OpenCV (cv2)`	4.x	Stereo matching, visualisation, and image processing. Key APIs: `StereoSGBM_create`, `compute`, `applyColorMap`, `convertScaleAbs`, `cvtColor`, `rectangle`, `putText`.
`NumPy`	2.x	Numerical array operations and the custom SAD implementation. Key APIs: `asanyarray`, `cumsum`, `full`, `zeros`, `abs`, `hstack`, `vstack`, `astype`.
`MediaPipe`	0.x	Deep learning inference for face detection. Key APIs: `tasks.vision.FaceDetector`, `FaceDetectorOptions`, `mp.Image`, `detector.detect`.

Model file property	Value
Filename	`blaze_face_full_range.tflite`
Size	≈1.1 MB
Format	TensorFlow Lite (flatbuffer)
Architecture	BlazeFace (full-range variant)
Detection range	20%–100% of image width
Confidence threshold	0.5 (configurable)
Input format	SRGB via `mp.Image` wrapper

10. How to Run

10.1. Hardware Requirements

- **Intel RealSense D455F** camera connected via USB 3.1 Gen 1 or faster. USB 2 is not supported for multi-stream capture at 30 FPS.
- A machine with a CPU capable of running OpenCV SGBM and NumPy SAD in real time (any modern quad-core is sufficient, though SAD will be slow).

10.2. System-Level Dependency: librealsense2

`pyrealsense2` is a Python wrapper around Intel's **librealsense2** C++ SDK, which must be installed at the OS level before the Python package will work. On Ubuntu/Debian:

```
# Register Intel's package server
sudo apt-key adv --keyserver keyserver.ubuntu.com \
     --recv-key F6E65AC044F831AC80A06380C8B3A55A6F3EFCDE
sudo add-apt-repository \
     "deb https://librealsense.intel.com/Debian/apt-repo $(
        lsb_release -cs) main"
sudo apt update
```

```

# Install SDK and utilities
sudo apt install -y librealsense2-dkms librealsense2-utils \
                    librealsense2-dev librealsense2-dbg
```

Listing 15: Install librealsense2 SDK (Ubuntu)

Verify the camera is detected before running the Python script:

```
rs-enumerate-devices   # should list the D455F and its streams
```

Listing 16: Verify camera connection

10.3. Python Environment Setup

```
# Create and activate a virtual environment (recommended)
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\
    activate

# Install all Python dependencies
pip install -r requirements.txt
```

Listing 17: Create environment and install dependencies

The `requirements.txt` pins the exact versions used during development:

Package	Version
`pyrealsense2`	2.56.5.9235
`numpy`	2.4.1
`opencv-python`	4.13.0.92
`mediapipe`	0.10.33

10.4. Project File Checklist

The following files must be present in the same directory before running:

File	Required	Notes
`live_stream.py`	Yes	Main application script
`blaze_face_full_range.tflite`	Yes	BlazeFace model; must be in the working directory
`requirements.txt`	Yes	Python dependency list

10.5. Running the Application

With the camera connected and the environment active, run:

```
python3 live_stream.py
```

Listing 18: Launch the live stream

A window titled **Stream Grid** will open displaying the 2×3 panel layout. Press **Q** or close the window to stop the application cleanly — the `finally` block in the script ensures `pipeline.stop()` is always called, releasing the camera.

Expected startup time: The first frame takes 1–3 seconds to appear while the camera pipeline initialises and the BlazeFace model is loaded into memory. The SAD depth panel will update noticeably slower than the other panels due to its sequential per-disparity computation over 128 levels.

10.6. Common Issues

Error	Fix
`RuntimeError: No device connected`	Check USB cable and port (must be USB 3). Run `rs-enumerate-devices` to confirm the camera is visible to the SDK.
`Failed to load model`	Ensure `blaze_face_full_range.tflite` is in the same directory as `live_stream.py`, or provide an absolute path.
`cv2.error: (-215) disparity`	IR frames must be grayscale (Y8). Confirm stream format is `rs.format.y8` in the config.
Blank / black depth panels	The IR dot projector may be disabled. Use `realsense-viewer` to confirm both IR streams are active and the projector is on.

11. Conclusion

This project demonstrates a complete real-time stereo vision pipeline for face depth estimation using the Intel RealSense D455F. Three independently implemented depth methods operate in parallel on the same hardware streams, providing a direct comparison of their characteristics.

- **Hardware ASIC depth** provides the best quality with the least code. It leverages Intel's proprietary stereo processor and requires only a single SDK call. It is the practical solution for real-world applications.
- **OpenCV SGBM** demonstrates how a classical computer vision algorithm can approach hardware-quality depth by incorporating global smoothness constraints through semi-global dynamic programming.
- **Custom SAD block matching** provides the clearest educational view of how stereo matching works at the fundamental level. Despite being the simplest and noisiest method, it illuminates the core principle behind all stereo vision: finding corresponding pixels across two viewpoints and measuring their horizontal displacement.

The integration of BlazeFace for lightweight real-time face detection, combined with the vectorised NumPy implementation of SAD (using the cumulative sum trick), illustrates how modern deep learning tools and classical optimisation techniques can coexist effectively in a single real-time system.

Possible extensions: applying WLS (Weighted Least Squares) post-filtering on SGBM disparities to reduce noise near edges; implementing temporal depth smoothing for face tracking; benchmarking all three methods against a known ground-truth distance; or replacing the SAD matching cost with the more robust Census transform.